

## JAVA 17 OCP · YDS STUDY SERIES

# Unit 13 · Concurrency — Practice Quiz

Teknik İngilizceyi bağlam içinde öğren · kısa tekrarlarla kalıcılaştır

Bu belge sekiz adet **OCF tarzı özgün çalışma sorusu** içerir; sorular gerçek sınavdan alınmamıştır. Önerilen süre 25–30 dakikadır; istersen 1–4 ve 5–8 olarak iki oturuma böl. Her soruda deterministic output, possible output, runtime exception ve compilation error ayrımını açıkça yap.

## Sorular

### Soru 1

**Odak:** Single-thread executor ve **Future**

Aşağıdaki programın çıktısı nedir?

```
import java.util.concurrent.Executors;

public class OneWorker {
    public static void main(String[] args) throws Exception {
        var service = Executors.newSingleThreadExecutor();
        try {
            var first = service.submit(() -> "A");
            var second = service.submit(() -> "B");
            System.out.print(first.get() + second.get());
        } finally {
            service.shutdown();
        }
    }
}
```

- A. Her zaman **AB**
- B. Her zaman **BA**
- C. **AB** veya **BA**
- D. Kod derlenmez.

### Soru 2

**Odak:** Java 17'de **ExecutorService**

Aşağıdaki kod parçası Java 17'de nasıl sonuçlanır?

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class ExecutorResource {
    public static void main(String[] args) {
        try (ExecutorService service =
            Executors.newSingleThreadExecutor()) {
            service.submit(() -> System.out.print("task"));
        }
    }
}
```

- A. **task** yazdırır ve executor otomatik kapanır.
- B. Runtime'da **RejectedExecutionException** oluşur.
- C. **Does not compile**; Java 17'de **ExecutorService**, **AutoCloseable** değildir.
- D. Kod derlenir fakat JVM hiçbir zaman sonlanmaz.

**Soru 3**

**Odak:** `volatile` ve atomicity

Birden fazla thread'in paylaştığı `volatile int counter` alanı için `counter++` kullanılıyor. Hangisi doğrudur?

- A. `volatile`, increment işlemini atomic yaptığı için update kaybı olmaz.
- B. `volatile` visibility sağlar; read-modify-write bütünü atomic olmadığı için lost update oluşabilir.
- C. `volatile`, yalnız reference type'larda kullanılabilir.
- D. Her `volatile` field erişimi aynı zamanda intrinsic monitor edinir.

**Soru 4**

**Odak:** Concurrent collection ve coordination

Aşağıdaki ifadelerden **hangi ikisi doğrudur?**

- A. `ConcurrentHashMap` iterator'ları weakly consistent olabilir.
- B. `CopyOnWriteArrayList` iterator'ı oluşturulduğu andaki snapshot üzerinden ilerler.
- C. `Collections.synchronizedList()` üzerinde iteration yapılırken ek synchronization hiçbir zaman gerekmez.
- D. Concurrent collection kullanmak bütün compound action'ları otomatik olarak atomic yapar.
- E. `CyclicBarrier`, party sayısına normal biçimde ulaştığı anda broken olur ve yeniden kullanılamaz.

**Soru 5**

**Odak:** Parallel stream ordering

Ordered bir stream üzerinde `parallel().findAny()` kullanıldığında hangi garanti vardır?

- A. Her zaman encounter order'daki ilk element döner.
- B. Uygun herhangi bir element dönebilir; ilk element garantisi yoktur.
- C. Parallel stream'de `findAny()` çağrısı compilation error üretir.
- D. Stream boşsa `null` döner.

**Soru 6**

**Odak:** English → Turkish / YDS

Aşağıdaki cümleyi doğal Türkçeye çevir ve `although` bağlacının kurduğu ilişkiyi belirt:

Although volatile guarantees visibility, it does not make a compound operation such as counter++ atomic.

**Soru 7**

**Odak:** Görevdeki exception ile `Future.get()` exception'ı

Aşağıdaki tam program için **tek doğru** sonuç hangisidir?

```
import java.io.IOException;
import java.util.concurrent.*;

public class FutureFailure {
    public static void main(String[] args) throws Exception {
        var service = Executors.newSingleThreadExecutor();
        try {
            Callable<Integer> task = () -> { throw new IOException("disk"); };
            var future = service.submit(task);
            try {
                future.get();
            } catch (ExecutionException e) {
                System.out.print(e.getCause().getClass().getSimpleName());
            }
        } finally {
            service.shutdown();
        }
    }
}
```

- A. Derlenmez; `Callable.call()` checked exception fırlatamaz.
- B. `IOException` yazdırır.
- C. `ExecutionException` yazdırır.
- D. `IOException`, `submit()` çağrısından doğrudan dışarı fırlatılır.

**Soru 8**

**Odak:** Koleksiyon güncellemesi ile iterator görünümü

Aşağıdaki programın **tek doğru** sonucunu seç.

```
import java.util.List;
import java.util.concurrent.CopyOnWriteArrayList;

public class SnapshotIteration {
    public static void main(String[] args) {
        var values = new CopyOnWriteArrayList<>(List.of("A", "B"));
        var iterator = values.iterator();
        values.add("C");
        iterator.forEachRemaining(System.out::print);
        System.out.print(":" + values.size());
    }
}
```

- A. `ABC:3`
- B. `AB:2`
- C. `AB:3`
- D. `ConcurrentModificationException` oluşur.

**Dil aktarımı:** “Although the list has changed, the iterator retains its snapshot.” cümlesinde `although` hangi beklenen çıkarımı geçersiz kılıyor?

## Cevaplar ve açıklamalar

### Soru 1 — A

- **A doğru:** Her `Future` kendi sabit `result`'ini taşır ve `+` expression'ı soldan sağa `first.get()`, ardından `second.get()` olarak değerlendirilir. Bu nedenle çıktı `AB`dir. `Single-thread executor` task'leri ayrıca `submission` sırasıyla `sequential` yürütür; ancak birleştirme sırasının determinist olması yalnız bu özelliğe bağlı değildir.
- **B yanlış:** `Print` expression `result`'ları `second`, sonra `first` sırasıyla birleştirmez.
- **C yanlış:** Task'lerin ne zaman tamamlandığından bağımsız olarak iki `Future` `result`'inin birleştirilme sırası deterministiktir.
- **D yanlış:** Lambda'lar `Callable<String>` olarak seçilir; checked `get()` exception'ları `main` tarafından declare edilmiştir.

### Soru 2 — C

- **A yanlış:** Bu davranış daha yeni API sürümleriyle karıştırılmamalıdır; Java 17 target'ında `ExecutorService` `try-with-resources` resource'u olamaz.
- **B yanlış:** Program runtime'a ulaşmaz.
- **C doğru:** Java 17'de `ExecutorService`, `AutoCloseable`ı extend etmez; compiler resource type'ını reddeder.
- **D yanlış:** Nontermination değerlendirmesinden önce compilation başarısız olur.

### Soru 3 — B

- **A yanlış:** `counter++` `read`, `add` ve `write` adımlarından oluşur; `volatile` bunları tek atomic action'a dönüştürmez.
- **B doğru:** Güncel değerin görülmesine yardım eder, fakat iki thread aynı eski değeri okuyup update'lardan birini kaybedebilir.
- **C yanlış:** Primitive ve reference field'lar `volatile` olabilir.
- **D yanlış:** `volatile` access monitor edinmez ve mutual exclusion sağlamaz.

### Soru 4 — A ve B

- **A doğru:** `ConcurrentHashMap` iterator'ı concurrent modification karşısında weakly consistent davranabilir.
- **B doğru:** Copy-on-write iterator, oluşturulduğu andaki backing array snapshot'ını görür.
- **C yanlış:** Synchronized wrapper üzerinde iteration sırasında wrapper object üzerinde ayrıca synchronization gerekir.
- **D yanlış:** Check-then-act gibi çok adımlı operation'lar uygun atomic API veya external coordination gerektirebilir.
- **E yanlış:** Normal trip sonrası `CyclicBarrier` yeni cycle için yeniden kullanılabilir; yalnız limite ulaşması onu broken yapmaz.

**Soru 5 — B**

- **A yanlış:** İlk element garantisi için `ordered stream`'de `findFirst()` kullanılır.
- **B doğru:** `findAny()` parallel execution'a daha fazla serbestlik verir; encounter order'daki ilk element zorunlu değildir.
- **C yanlış:** `findAny()` hem sequential hem parallel stream API'sinde geçerlidir.
- **D yanlış:** Boş stream için `Optional.empty()` döner, `null` değil.

**Soru 6 — Örnek çeviri**

“`volatile` görünürlüğü garanti etse de `counter++` gibi birleşik bir işlemi atomik hâle getirmez.”

`although`, ilk önermeye rağmen ikinci önermenin geçerli olduğunu gösteren concession (ödünleme) bağlacıdır. YDS'de `although + clause` sonrasında ayrıca `but` kullanılmaz. `such as` örnek verir; `does not make A B` ise “A'yı B hâline getirmez” yapısıdır.

**Soru 7 — B**

- **B doğru:** `Callable` görevinin `IOException` hatası, `Future.get()` sırasında `ExecutionException` içine sarılır. Kod `getCause()` ile asıl hatanın sınıf adını okur; `IOException` yazar.
- **A yanlış:** `Callable.call()` checked exception bildirebilir; program Java 17'de derlenir.
- **C yanlış:** Yakalanan tür `ExecutionException` olsa da yazdırılan onun nedeni olan nesnenin türüdür.
- **D yanlış:** Görevin hatası, `submit()` çağrısından doğrudan çağırılan iş parçacığına taşınmaz; sonuç beklenirken gözlemlenir.

**Soru 8 — C**

- **C doğru:** İterator, oluşturulduğunda `A`, `B` anlık görüntüsünü tutar; liste ise artık üç öğelidir. Başarıyla derlenir ve `AB:3` yazdırır.
- **A yanlış:** Sonradan eklenen `C`, mevcut iterator'ın görüntüsüne girmez.
- **B yanlış:** Anlık görüntü, asıl listenin ekleme işlemini engellemez; `size()` 3 döndürür.
- **D yanlış:** `CopyOnWriteArrayList` iterator'ı bu değişiklik nedeniyle `ConcurrentModificationException` fırlatmaz.

Çeviri: “Liste değişmiş olsa da iterator, kendi anlık görüntüsünü korur.” `although`, listenin değişmesinin mevcut iterator görünümünü de değiştireceği beklentisini geçersiz kılar.